CSCI-4208: Developing Advanced Web Applications

Week 8: Lecture 12

JavaScript - Asynchronous

Overview: Asynchronous JS

- intro
- promises
- fetch
- then
- async/await
- axios

Promise

A **`Promise`** is a proxy for a value not necessarily known when the promise is created. It allows you to associate handlers with an asynchronous action's eventual success value or failure reason. This lets asynchronous methods return values like synchronous methods: instead of immediately returning the final value, the asynchronous method returns a *promise* to supply the value at some point in the future.

A `Promise` is in one of these states:

- *pending*: initial state, neither fulfilled nor rejected.
- *fulfilled*: meaning that the operation was completed successfully.
- *rejected*: meaning that the operation failed.

Promise

The constructor syntax for a promise object is:

```
let promise = new Promise(function(resolve, reject) {
  // executor (the producing code)
});
```

The function passed to `new Promise` is called the *executor*. When `new Promise` is created, the executor runs automatically. It contains the producing code which should eventually produce the result.

Its arguments `resolve` and `reject` are callbacks provided by JavaScript itself. Our code is only inside the executor.

When the executor obtains the result, be it soon or late, doesn't matter, it should call one of these callbacks:

- `resolve(value)` — if the job finished successfully, with result `value`.
- `reject(error)` — if an error occurred, `error` is the error object.

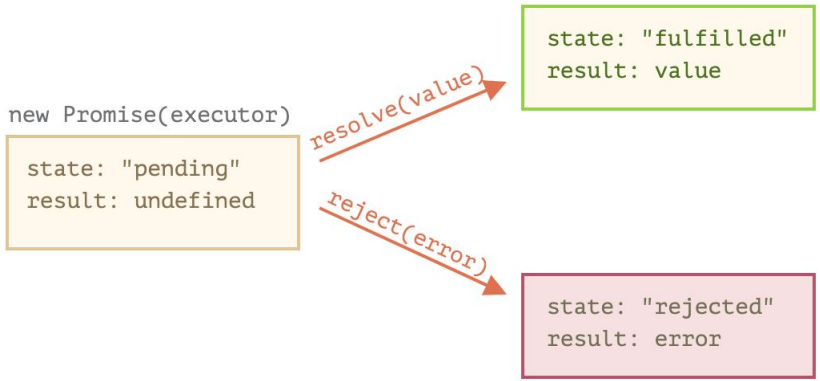
Promise

So to summarize: the executor runs automatically and attempts to perform a job. When it is finished with the attempt it calls `resolve` if it was successful or `reject` if there was an error.

The `promise` object returned by the `new Promise` constructor has these internal properties:

- `state` — initially `"pending"`, then changes to either `"fulfilled"` when `resolve` is called or `"rejected"` when `reject` is called.
- `result` — initially `undefined`, then changes to `value` when `resolve(value)` called or `error` when `reject(error)` is called.

So the executor eventually moves `promise` to one of these states:



Promise

Here's an example of a promise constructor and a simple executor function with "producing code" that takes time (via `setTimeout`):

```
let promise = new Promise(function(resolve, reject) {
    // the function is executed automatically when the promise is constructed

    // after 1 second signal that the job is done with the result "done"
    setTimeout(() => resolve("done"), 1000);
});
```

We can see two things by running the code above:

1. The executor is called automatically and immediately (by `new Promise`).
2. The executor receives two arguments: `resolve` and `reject`. These functions are predefined by the JavaScript engine, so we don't need to create them. We should only call one of them when ready.
   After one second of "processing" the executor calls `resolve("done")` to produce the result. This changes the state of the `promise` object:

Promise

That was an example of a successful job completion, a “fulfilled promise”.

And now an example of the executor rejecting the promise with an error:

```
let promise = new Promise(function(resolve, reject) {
  // after 1 second signal that the job is finished with an error
  setTimeout(() => reject(new Error("Whoops!")), 1000);
});
```

The call to `reject(...)` moves the promise object to `"rejected"` state:

To summarize, the executor should perform a job (usually something that takes time) and then call `resolve` or `reject` to change the state of the corresponding promise object.

A promise that is either resolved or rejected is called “settled”, as opposed to an initially “pending” promise.

Consumers: then, catch, finally

A Promise object serves as a link between the executor (the “producing code”) and the consuming functions, which will receive the result or error. Consuming functions can be registered (subscribed) using methods `.then`, `.catch` and `.finally`.

then

The syntax is:

```
promise.then(
  function(result) { /* handle a successful result */ },
  function(error) { /* handle an error */ }
);
```

The first argument of `.then` is a function that runs when the promise is resolved, and receives the result.

The second argument of `.then` is a function that runs when the promise is rejected, and receives the error.

Consumers: then, catch, finally

For instance, here’s a reaction to a successfully resolved promise:

```
let promise = new Promise(resolve => {
  setTimeout(() => resolve("done!"), 1000);
});

promise.then(alert); // shows "done!" after 1 second
```

Consumers: then, catch, finally

catch

If we're interested only in errors, then we can use `null` as the first argument: `.then(null, errorHandlingFunction)`. Or we can use `.catch(errorHandlingFunction)`, which is exactly the same:

```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => reject(new Error("Whoops!")), 1000);
});

// .catch(f) is the same as promise.then(null, f)
promise.catch(alert); // shows "Error: Whoops!" after 1 second
```

The call `.catch(f)` is a complete analog of `.then(null, f)`, it's just a shorthand.

Consumers: then, catch, finally

finally

Just like there's a `finally` clause in a regular `try {...} catch {...}`, there's `finally` in promises.

The call `.finally(f)` is similar to `.then(f, f)` in the sense that `f` always runs when the promise is settled: be it resolve or reject.

`finally` is a good handler for performing cleanup, e.g. stopping our loading indicators, as they are not needed anymore, no matter what the outcome is.

```
new Promise((resolve, reject) => {
  /* do something that takes time, and then call resolve/reject */
})
  // runs when the promise is settled, doesn't matter successfully or not
  .finally(() => stop loading indicator)
  // so the loading indicator is always stopped before we process the result/error
  .then(result => show result, err => show error)
```

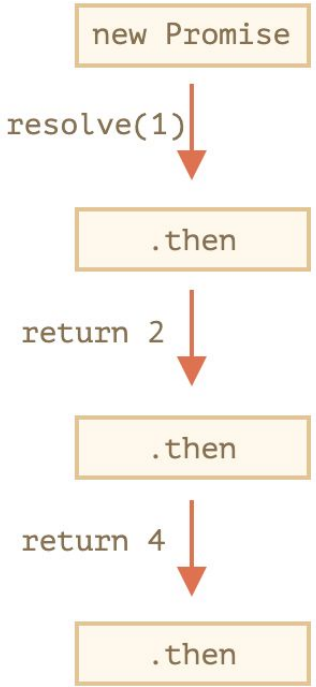
Promises chaining

We have a sequence of asynchronous tasks to be performed one after another —

Promises provide a means to handle this

```
new Promise(function(resolve, reject) {
  setTimeout(() => resolve(1), 1000); // (*)

}).then(function(result) { // (**)

  alert(result); // 1
  return result * 2;

}).then(function(result) { // (***)

  alert(result); // 2
  return result * 2;

}).then(function(result) {

  alert(result); // 4
  return result * 2;

});
```

Promises chaining



The idea is that the result is passed through the chain of `.then` handlers.

Here the flow is:

1. The initial promise resolves in 1 second `(*)`,
2. Then the `.then` handler is called `(**)`.
3. The value that it returns is passed to the next `.then` handler `(***)`
4. ...and so on.

As the result is passed along the chain of handlers,
we can see a sequence of `alert` calls: `1` → `2` → `4`.

The whole thing works, because a call to `promise.then` returns a promise, so that we can call the next `.then` on it.

When a handler returns a value, it becomes the result of that promise, so the next `.then` is called with it.

Returning promises

A handler, used in `.then(handler)` may create and return a promise. In that case further handlers wait until it settles, and then get its result.

```
new Promise(function(resolve, reject) {

  setTimeout(() => resolve(1), 1000);

}).then(function(result) {

  alert(result); // 1

  return new Promise((resolve, reject) => { // (*)
    setTimeout(() => resolve(result * 2), 1000);
  });

}).then(function(result) { // (**)

  alert(result); // 2

  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(result * 2), 1000);
  });

}).then(function(result) {

  alert(result); // 4

});
```

Error handling with promises

Promise chains are great at error handling. When a promise rejects, the control jumps to the closest rejection handler. That's very convenient in practice.

For instance, in the code below the URL to `fetch` is wrong (no such site) and `.catch` handles the error:

```
fetch('https://no-such-server.blabla') // rejects
  .then(response => response.json())
  .catch(err => alert(err)) // TypeError: failed to fetch (the text may vary)
```

As you can see, the `.catch` doesn't have to be immediate. It may appear after one or maybe several `.then`.

Promise API

There are 6 static methods of `Promise` class:

1. `Promise.all(promises)` – waits for all promises to resolve and returns an array of their results. If any of the given promises rejects, it becomes the error of `Promise.all`, and all other results are ignored.
2. `Promise.allSettled(promises)` (recently added method) – waits for all promises to settle and returns their results as an array of objects with:
   - `status`: `"fulfilled"` or `"rejected"`
   - `value` (if fulfilled) or `reason` (if rejected).
3. `Promise.race(promises)` – waits for the first promise to settle, and its result/error becomes the outcome.
4. `Promise.any(promises)` (recently added method) – waits for the first promise to fulfill, and its result becomes the outcome. If all of the given promises are rejected, `AggregateError` becomes the error of `Promise.any`.
5. `Promise.resolve(value)` – makes a resolved promise with the given value.
6. `Promise.reject(error)` – makes a rejected promise with the given error.

Of all these, `Promise.all` is probably the most common in practice.

Promise API - Promise All

Promise.all

Let’s say we want many promises to execute in parallel and wait until all of them are ready.

For instance, download several URLs in parallel and process the content once they are all done.

That’s what `Promise.all` is for.

The syntax is:

```
let promise = Promise.all([...promises...]);
```

`Promise.all` takes an array of promises (it technically can be any iterable, but is usually an array) and returns a new promise.

The new promise resolves when all listed promises are settled, and the array of their results becomes its result.

Promise API - Promise All

For instance, the `Promise.all` below settles after 3 seconds, and then its result is an array `[1, 2, 3]`:

```
Promise.all([
  new Promise(resolve => setTimeout(() => resolve(1), 3000)), // 1
  new Promise(resolve => setTimeout(() => resolve(2), 2000)), // 2
  new Promise(resolve => setTimeout(() => resolve(3), 1000))  // 3
]).then(alert); // 1,2,3 when promises are ready: each promise contributes an array member
```

Promise API - All Settled

`Promise.all` rejects as a whole if any promise rejects. That's good for "all or nothing" cases, when we need *all* results successful to proceed:

```
Promise.all([
  fetch('/template.html'),
  fetch('/style.css'),
  fetch('/data.json')
]).then(render); // render method needs results of all fetches
```

`Promise.allSettled` just waits for all promises to settle, regardless of the result. The resulting array has:

- `{status:"fulfilled", value:result}` for successful responses,
- `{status:"rejected", reason:error}` for errors.

Promise API - race

Similar to `Promise.all`, but waits only for the first settled promise and gets its result (or error).

The syntax is:

```
let promise = Promise.race(iterable);
```

For instance, here the result will be `1`:

```
Promise.race([
  new Promise((resolve, reject) => setTimeout(() => resolve(1), 1000)),
  new Promise((resolve, reject) => setTimeout(() => reject(new Error("Whoops!")), 2000)),
  new Promise((resolve, reject) => setTimeout(() => resolve(3), 3000))
]).then(alert); // 1
```

The first promise here was fastest, so it became the result. After the first settled promise “wins the race”, all further results/errors are ignored.

Promise API - any

Similar to `Promise.race`, but waits only for the first fulfilled promise and gets its result. If all of the given promises are rejected, then the returned promise is rejected with `AggregateError` – a special error object that stores all promise errors in its `errors` property.

The syntax is:

```
let promise = Promise.any(iterable);
```

For instance, here the result will be `1`:

```
Promise.any([
  new Promise((resolve, reject) => setTimeout(() => reject(new Error("Whoops!")), 1000)),
  new Promise((resolve, reject) => setTimeout(() => resolve(1), 2000)),
  new Promise((resolve, reject) => setTimeout(() => resolve(3), 3000))
]).then(alert); // 1
```

The first promise here was fastest, but it was rejected, so the second promise became the result. After the first fulfilled promise “wins the race”, all further results are ignored.

Promise API - resolve

`Promise.resolve(value)` creates a resolved promise with the result `value`.

Same as:

```
let promise = new Promise(resolve => resolve(value));
```

The method is used for compatibility, when a function is expected to return a promise.

Promise API - reject

`Promise.reject(error)` creates a rejected promise with `error`.

Same as:

```
let promise = new Promise((resolve, reject) => reject(error));
```

In practice, this method is almost never used.

Promise API - Summary

There are 6 static methods of `Promise` class:

1. `Promise.all(promises)` – waits for all promises to resolve and returns an array of their results. If any of the given promises rejects, it becomes the error of `Promise.all`, and all other results are ignored.
2. `Promise.allSettled(promises)` (recently added method) – waits for all promises to settle and returns their results as an array of objects with:
    - `status`: `"fulfilled"` or `"rejected"`
    - `value` (if fulfilled) or `reason` (if rejected).
3. `Promise.race(promises)` – waits for the first promise to settle, and its result/error becomes the outcome.
4. `Promise.any(promises)` (recently added method) – waits for the first promise to fulfill, and its result becomes the outcome. If all of the given promises are rejected, `AggregateError` becomes the error of `Promise.any`.
5. `Promise.resolve(value)` – makes a resolved promise with the given value.
6. `Promise.reject(error)` – makes a rejected promise with the given error.

Of all these, `Promise.all` is probably the most common in practice.

Async/Await

There's a special syntax to work with promises in a more comfortable fashion, called "async/await". It's surprisingly easy to understand and use.

Async functions

Let's start with the `async` keyword. It can be placed before a function, like this:

```
async function f() {
  return 1;
}
```

The word "async" before a function means one simple thing: a function always returns a promise. Other values are wrapped in a resolved promise automatically.

Async/Await

For instance, this function returns a resolved promise with the result of `1`; let's test it:

```
async function f() {
  return 1;
}

f().then(alert); // 1
```

...We could explicitly return a promise, which would be the same:

```
async function f() {
  return Promise.resolve(1);
}

f().then(alert); // 1
```

So, `async` ensures that the function returns a promise, and wraps non-promises in it. Simple enough, right? But not only that. There's another keyword, `await`, that works only inside `async` functions, and it's pretty cool.

Async/Await

Await

The syntax:

```
// works only inside async functions
let value = await promise;
```

The keyword `await` makes JavaScript wait until that promise settles and returns its result.

Here's an example with a promise that resolves in 1 second:

```
async function f() {
  let promise = new Promise((resolve, reject) => {
    setTimeout(() => resolve("done!"), 1000)
  });
  let result = await promise; // wait until the promise resolves (*)
  alert(result); // "done!"
}

f();
```

The function execution "pauses" at the line `(*)` and resumes when the promise settles, with `result` becoming its result.

`await` literally suspends the function execution until the promise settles, and then resumes it with the promise result.

It's a more elegant syntax of getting the promise result.

fetch *(async, promise-based)*

In frontend development, promises are often used for network requests. The fetch method sends requests to a server.

The basic syntax:

```
let promise = fetch(url);
```

This makes a network request to the `url` and returns a promise. The promise resolves with a `response` object when the remote server responds with headers, but *before the full response is downloaded*.

To read the full response, we should call the method `response.text()`: it returns a promise that resolves when the full text is downloaded from the remote server, with that text as a result.

The End.

Coming Soon... to a Moodle near you!

Labs	Quiz Game with Leaderboard
Homework	REST Client App using Modules pattern